

Raport

Przetwarzanie i analiza danych przestrzennych

Oracle spatial

Imiona i nazwiska:

Karolina Węgrzyn, Patrycja Markiewicz

Celem ćwiczenia jest zapoznanie się ze sposobem przechowywania, przetwarzania i analizy danych przestrzennych w bazach danych (na przykładzie systemu Oracle spatial)

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki, zrzut ekranu, komentarz

-- ...

Do wykonania ćwiczenia (zadania 1 – 6) i wizualizacji danych wykorzystaj Oracle SQL Developer. Alternatywnie możesz wykonać analizy w środowisku Python/Jupyter Notebook

Do wykonania zadania 7 wykorzystaj środowisko Python/Jupyter Notebook

Raport należy przesłać w formacie pdf.

Należy też dołączyć raport zawierający kod w formacie źródłowym.

Np.

- plik tekstowy .sql z kodem poleceń
- plik .md zawierający kod wersji tekstowej
- notebook programu jupyter – plik .ipynb

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

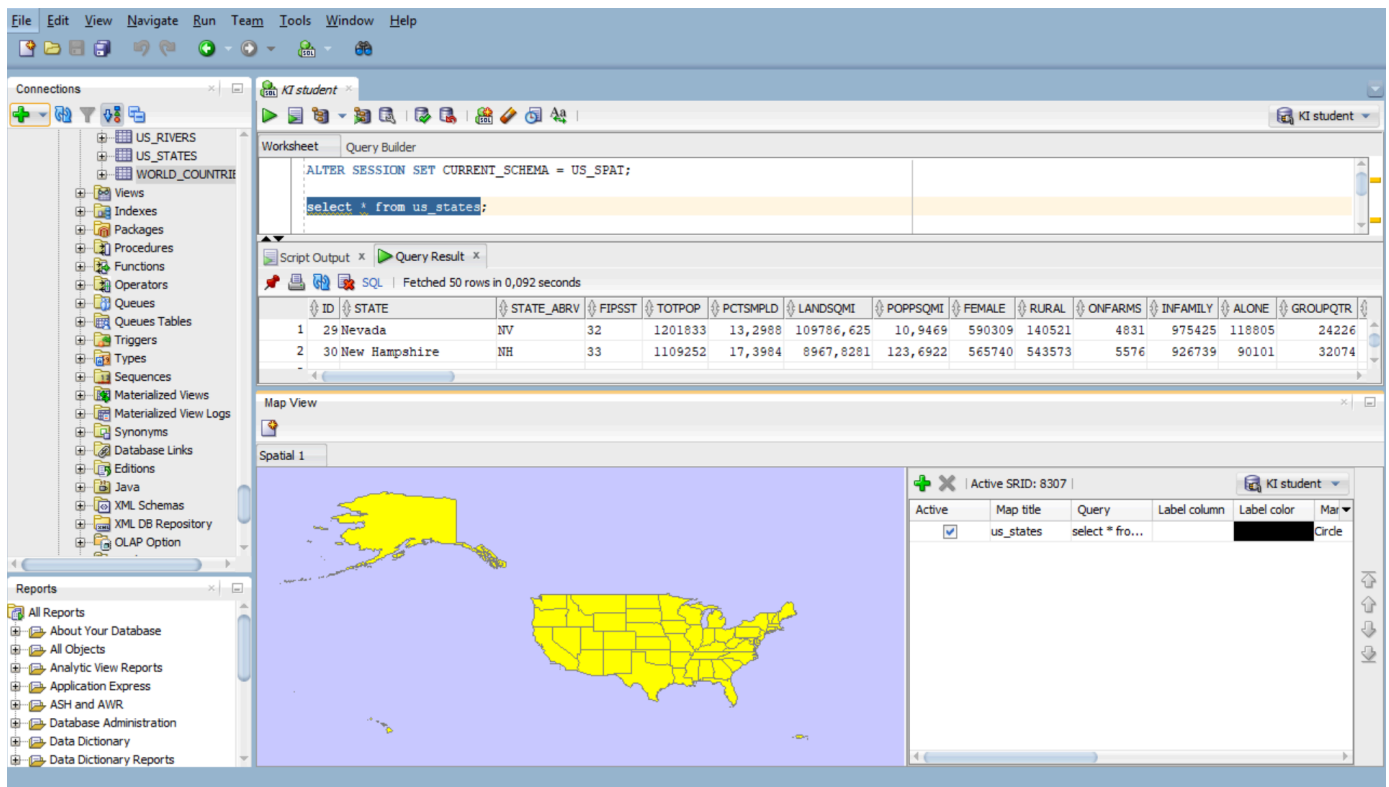
Zadanie 1

Zwizualizuj przykładowe dane

US_STATES

Wyniki, zrzut ekranu, komentarz

```
select * from us_states
```



US_INTERSTATES

Wyniki, zrzut ekranu, komentarz

```
select * from us_interstates
```

The top screenshot shows the Oracle SQL Developer interface with the following components:

- Connections:** A tree view on the left showing database connections like US_RIVERS, US_STATES, and WORLD_COUNTRIES.
- Worksheet:** Contains SQL queries:


```
ALTER SESSION SET CURRENT_SCHEMA = US_SPAT;
select * from us_states;
select * from world_countries;
```
- Script Output / Query Result:** Displays a table with 10 columns: ID, FIPS_CNTRY, GMI_CNTRY, ISO_2DIGIT, ISO_3DIGIT, CNTRY_NAME, SOVEREIGN, POP_CNTRY, SQKM_CNTRY, SQMI_CNTRY, and CURR. It shows two rows of data for Cayman Islands and Cocos (Keeling) Islands.
- Map View:** A map titled 'Spatial 1' showing a network of roads in a light blue area. A legend on the right shows two entries: 'us_states' with a black circle and 'US_INTERS...' with a black circle.

The bottom screenshot shows the same interface but with a different map view:

- Map View:** The map titled 'Spatial 1' now shows a network of roads in a yellow area, indicating a different spatial context or data source.
- Legend:** The legend on the right remains the same, showing 'us_states' and 'US_INTERS...' with black circles.

US_CITIES

Wyniki, zrzut ekranu, komentarz

```
select * from us_cities
```

The screenshot shows the KI student software interface. The left sidebar contains a tree view of database objects including US_RIVERS, US_STATES, WORLD_COUNTRIES, Views, Indexes, Packages, Procedures, Functions, Operators, Queues, Queues Tables, Triggers, Types, Sequences, Materialized Views, Materialized View Logs, Synonyms, Database Links, Editions, Java, XML Schemas, XML DB Repository, and OLAP Option. The main window is divided into several panes:

- Worksheet:** Contains SQL queries:


```
ALTER SESSION SET CURRENT_SCHEMA = US_SPAT;
select * from us_states;
select * from world_countries;
```
- Script Output / Query Result:** Displays a table with 10 columns: ID, FIPS_CNTRY, GMI_CNTRY, ISO_2DIGIT, ISO_3DIGIT, CNTRY_NAME, SOVEREIGN, POP_CNTRY, SQKM_CNTRY, SQMI_CNTRY, and CURR. It shows two rows of data for Cayman Islands and Cocos (Keeling) Islands.
- Map View:** Shows a map of the United States with red dots representing city locations. The map is titled 'Spatial 1'.
- Map View Properties:** A table on the right side of the map view shows the active SRID (8307) and a list of map elements:

Active	Map title	Query	Label column	Label color	Mar
☒	us_states	select * fro...			Circle
☒	US_INTERS...	select * fro...			Circle
☒	US_CITIES	select * fro...			Circle

```
select * from us_cities
where state_abrv = 'FL'
```

The screenshot shows the KI student software interface with a different query and map view. The left sidebar is the same as in the previous screenshot. The main window is divided into several panes:

- Worksheet:** Contains SQL queries:

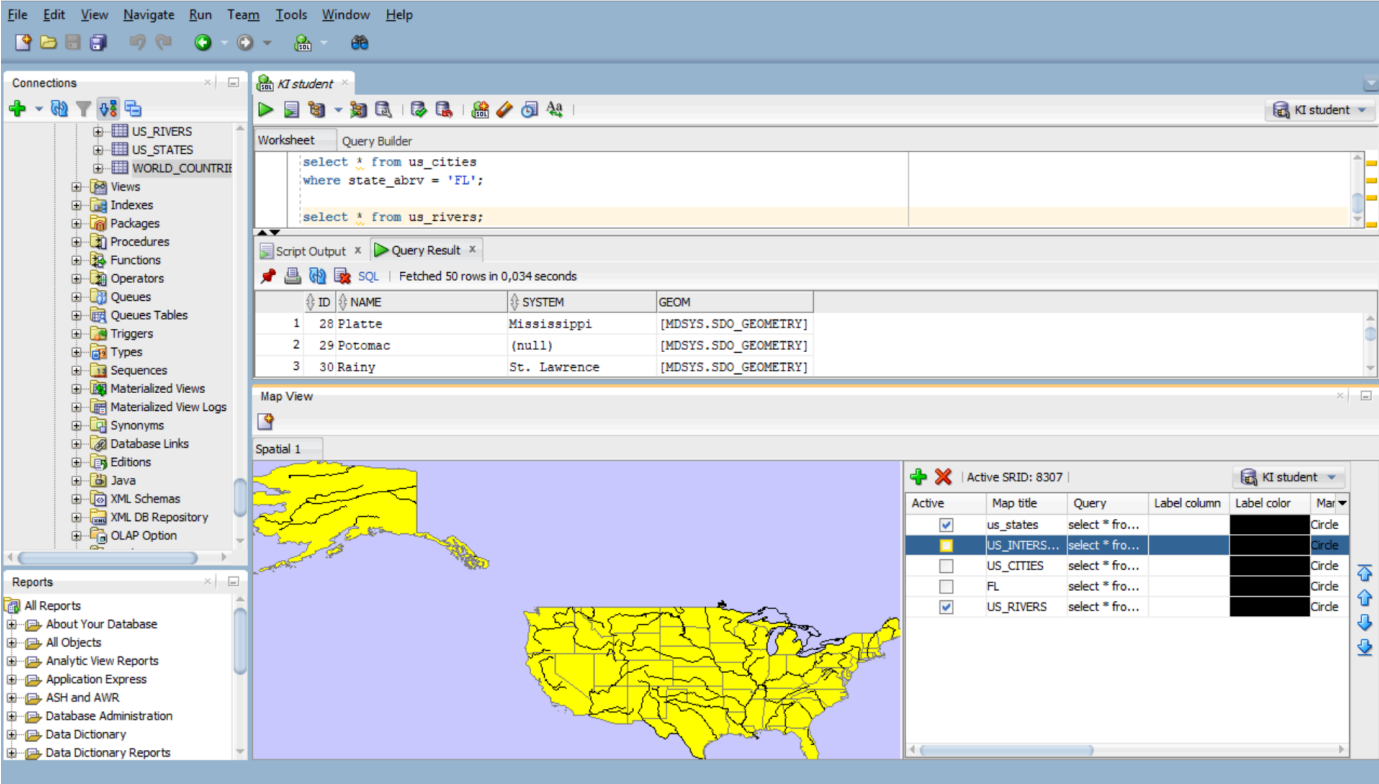

```
select * from world_countries;
select * from us_cities
where state_abrv = 'FL';
```
- Script Output / Query Result:** Displays a table with 6 columns: ID, CITY, STATE_ABRV, POP90, RANK90, and LOCATION. It shows three rows of data for Tallahassee, Hollywood, and Jacksonville in Florida.
- Map View:** Shows a map of Florida with red dots representing city locations. The map is titled 'Spatial 1'.
- Map View Properties:** A table on the right side of the map view shows the active SRID (8307) and a list of map elements:

Active	Map title	Query	Label column	Label color	Mar
☒	us_states	select * fro...			Circle
☒	US_INTERS...	select * fro...			Circle
☒	US_CITIES	select * fro...			Circle
☒	FL	select * fro...			Circle

US_RIVERS

Wyniki, zrzut ekranu, komentarz

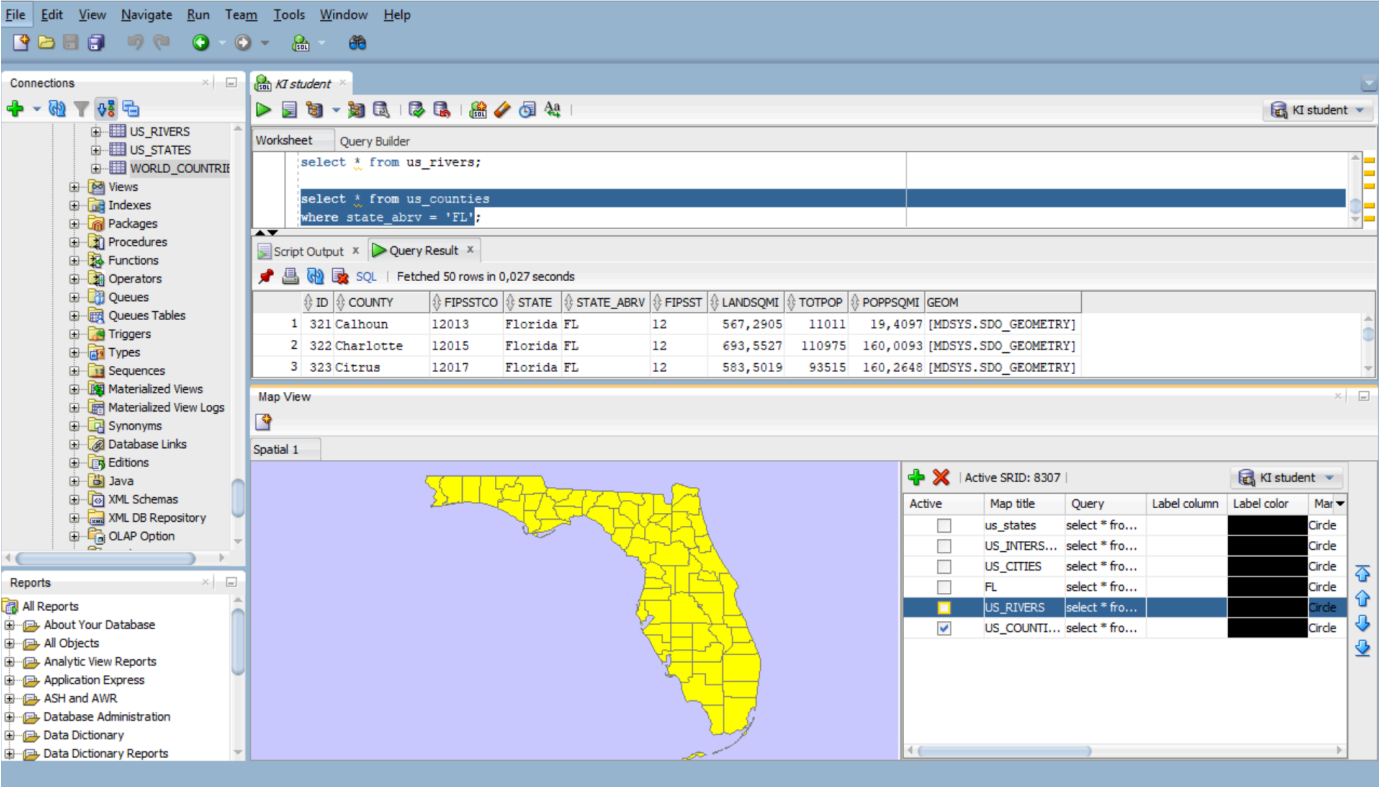
```
select * from us_rivers
```

US_COUNTIES

Wyniki, zrzut ekranu, komentarz

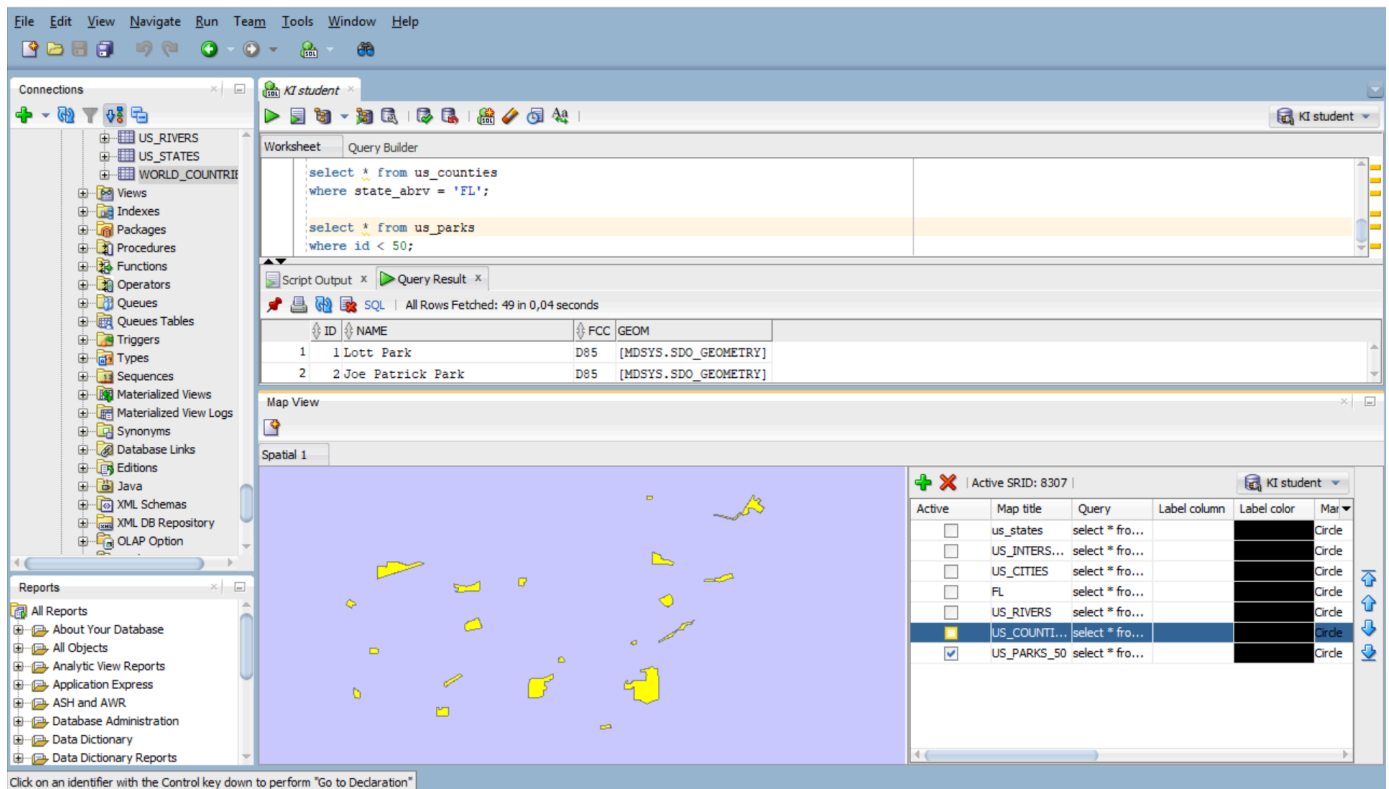
```
select * from us_counties
```



US_PARKS

Wyniki, zrzut ekranu, komentarz

```
select * from us_parks
where id < 50
```



Zadanie 2

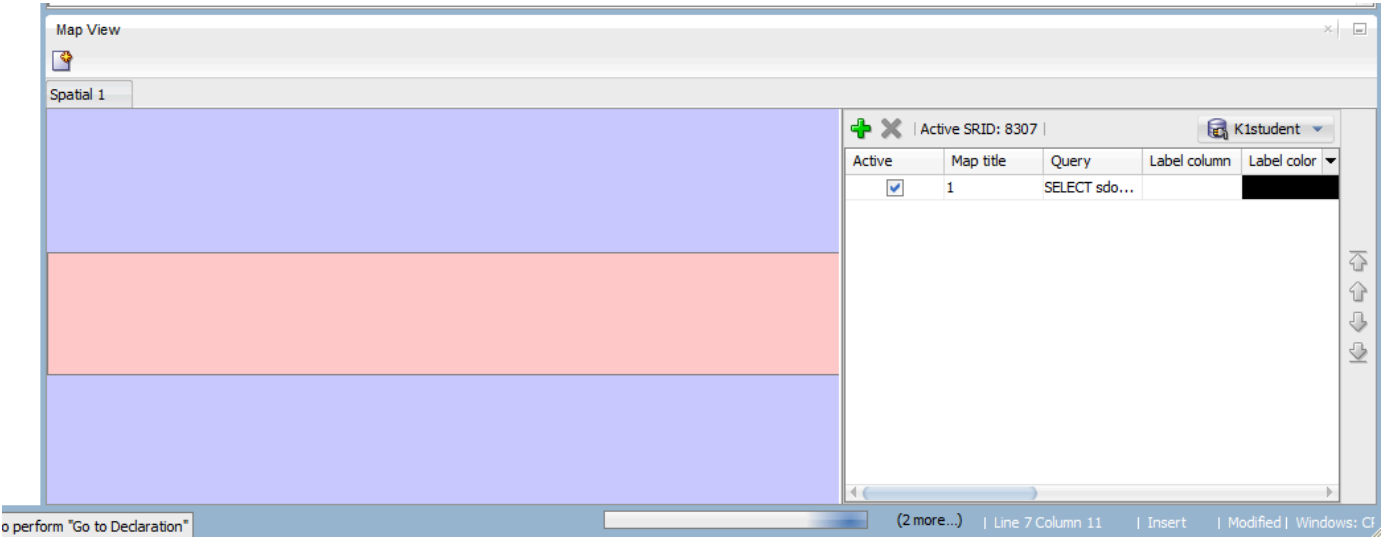
Znajdź wszystkie stany (us_states) których obszary mają część wspólną ze wskazaną geometrią (prostokątem)

Pokaż wynik na mapie.

prostokąt

```
SELECT sdo_geometry (2003, 8307, null,
sdo_elem_info_array (1,1003,3),
sdo_ordinate_array ( -117.0, 40.0, -90., 44.0)) g
FROM dual
```

Wyniki, zrzut ekranu, komentarz

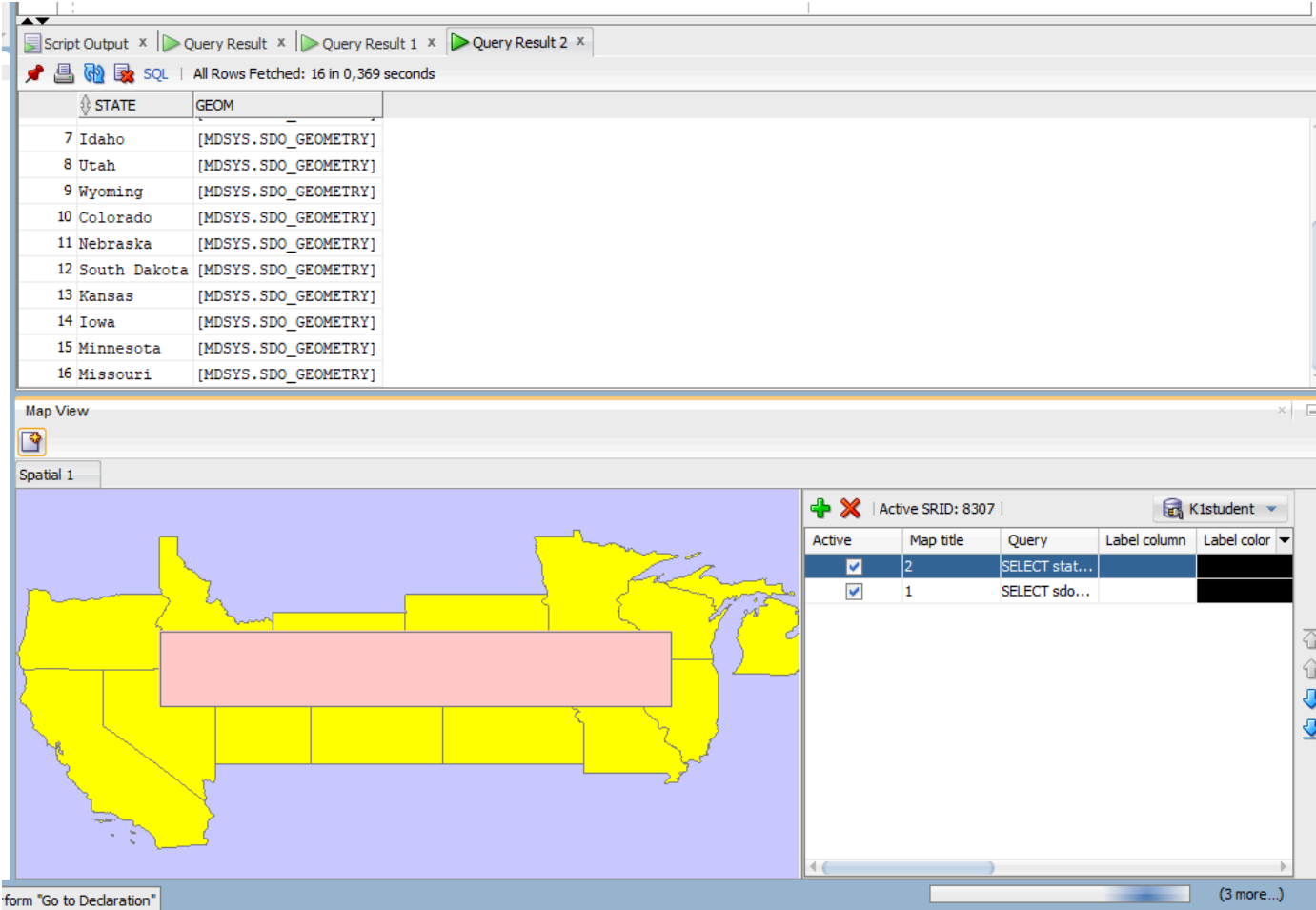


Użyj funkcji SDO_FILTER

```
SELECT state, geom FROM us_states
WHERE sdo_filter (geom,
sdo_geometry (2003, 8307, null,
sdo_elem_info_array (1,1003,3),
sdo_ordinate_array ( -117.0, 40.0, -90., 44.0))
) = 'TRUE';
```

Zwróć uwagę na liczbę zwróconych wierszy (16)

Wyniki, zrzut ekranu, komentarz



SDO_FILTER zwraca wszystkie obiekty, których minimalny prostokąt (MBR) przecina się z podanym prostokątem - daje wynik szybciej, ale mniej dokładnie. W efekcie zwraca więcej stanów, w tym przypadku 16.

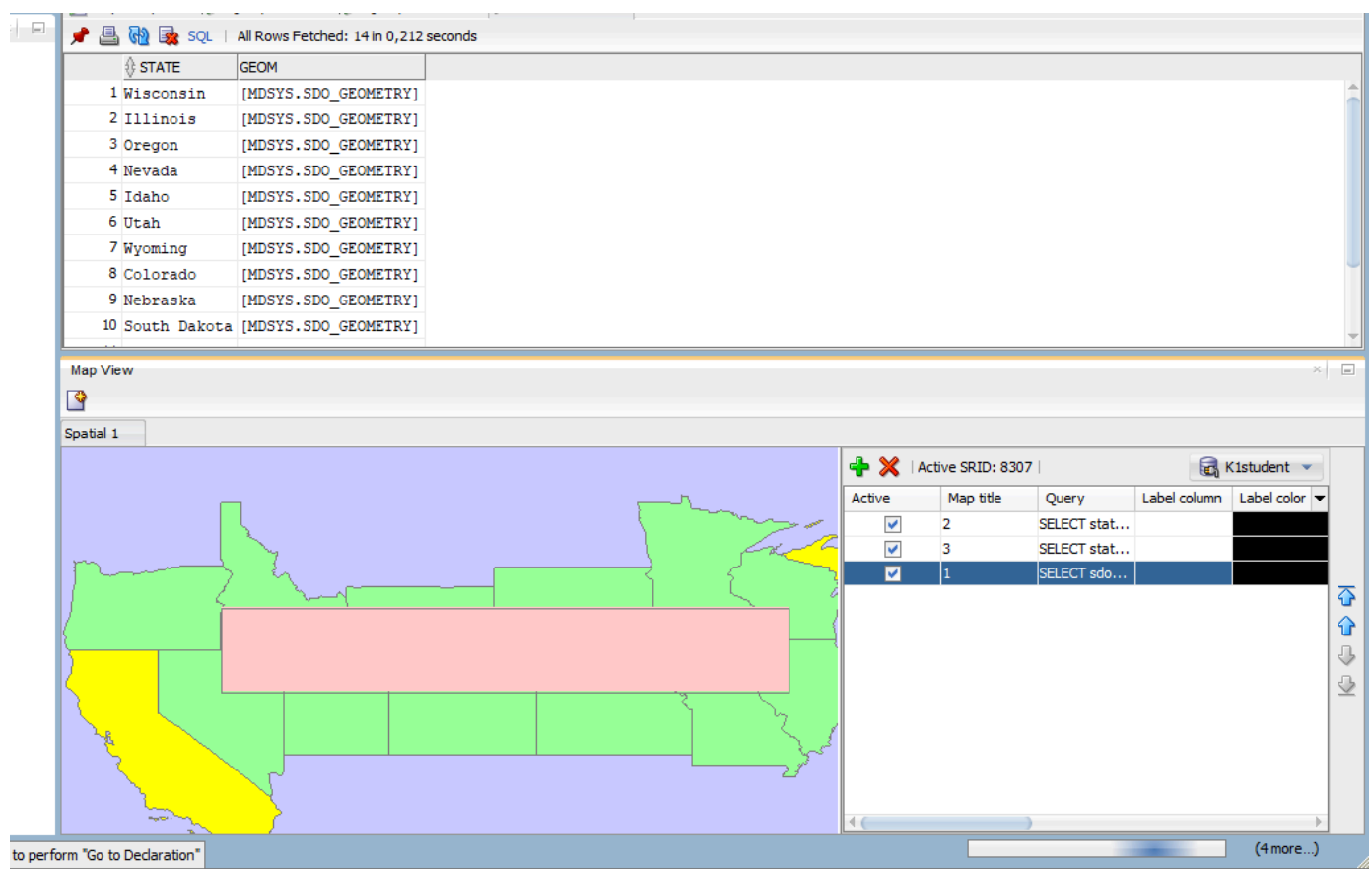
Użyj funkcji SDO_ANYINTERACT

```
SELECT state, geom FROM us_states
WHERE sdo_anyinteract (geom,
sdo_geometry (2003, 8307, null,
sdo_elem_info_array (1,1003,3),
sdo_ordinate_array ( -117.0, 40.0, -90., 44.0))
) = 'TRUE';
```

Porównaj wyniki sdo_filter i sdo_anyinteract

Pokaż wynik na mapie

Wyniki, zrzut ekranu, komentarz



SDO_ANYINTERACT sprawdza rzeczywiste przecięcie geometrii, więc zwraca dokładniejsze wyniki - tylko te stany, które faktycznie mają kontakt z prostokątem. Jest ich mniej (14) niż dla SDO_FILTER w tym przykładzie.

Zadanie 3

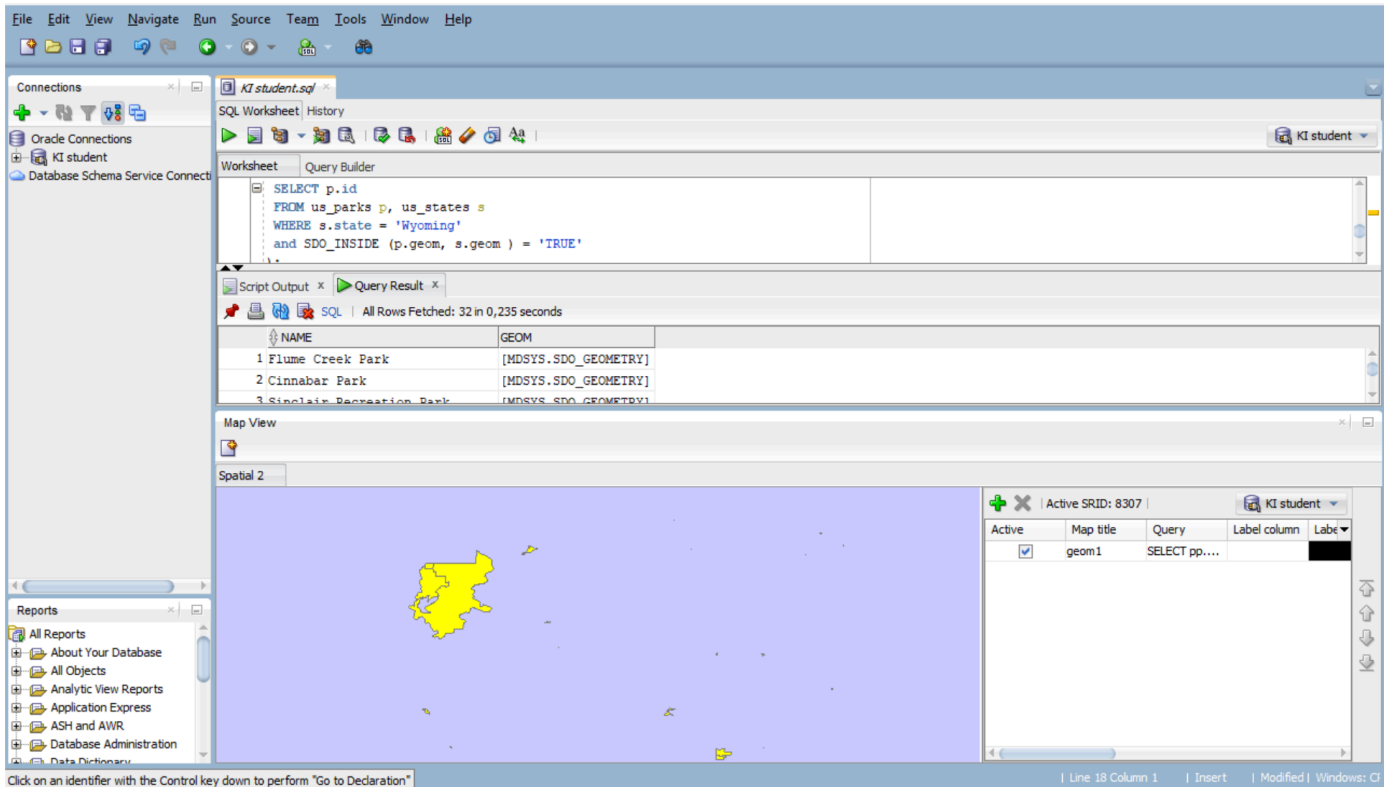
Znajdź wszystkie parki (us_parks) których obszary znajdują się wewnątrz stanu Wyoming

Użyj funkcji SDO_INSIDE

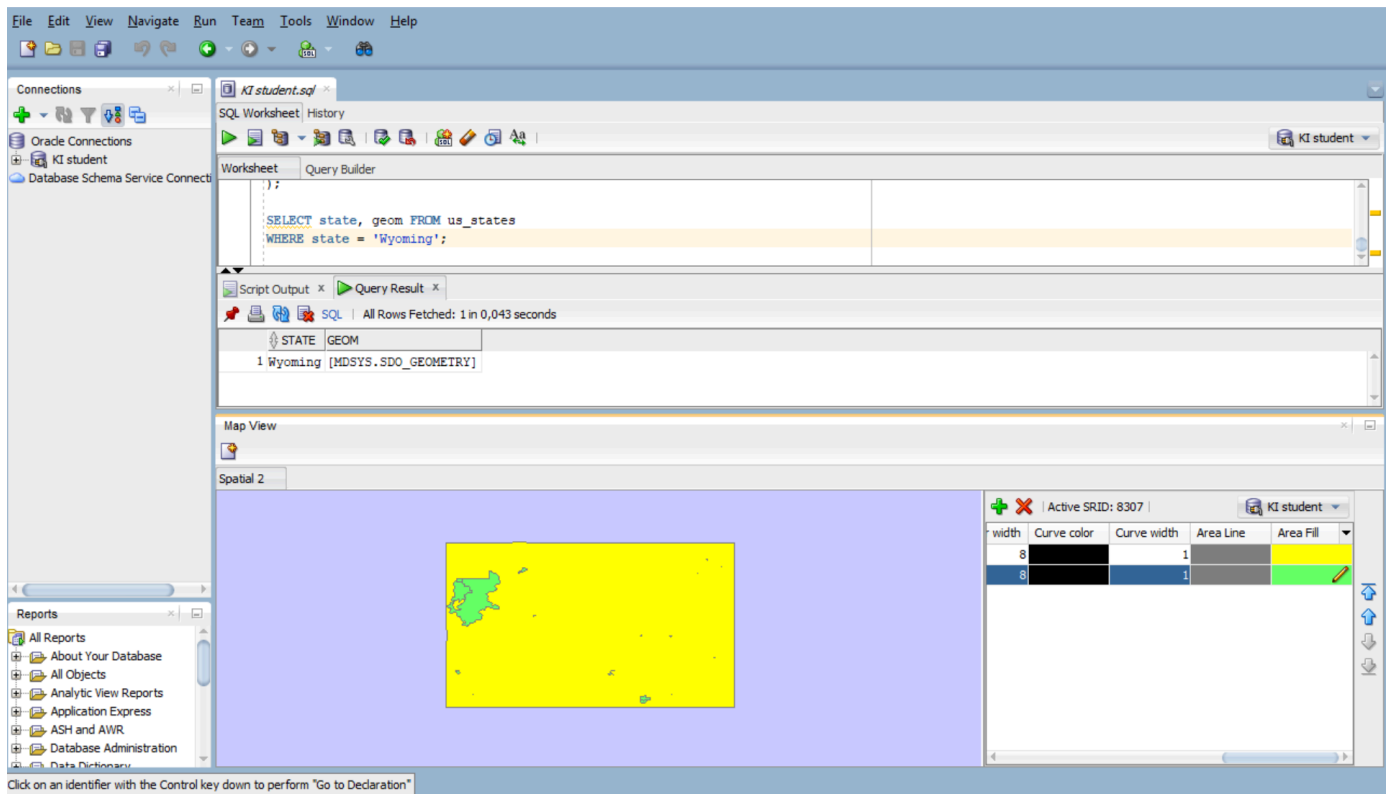
```
SELECT p.name, p.geom
FROM us_parks p, us_states s
WHERE s.state = 'Wyoming'
AND SDO_INSIDE (p.geom, s.geom) = 'TRUE';
```

W przypadku wykorzystywania narzędzia SQL Developer, w celu wizualizacji na mapie użyj podzapytania

```
SELECT pp.name, pp.geom FROM us_parks pp
WHERE id IN
(
    SELECT p.id
    FROM us_parks p, us_states s
    WHERE s.state = 'Wyoming'
        AND SDO_INSIDE (p.geom, s.geom ) = 'TRUE'
)
```



```
SELECT state, geom FROM us_states
WHERE state = 'Wyoming'
```

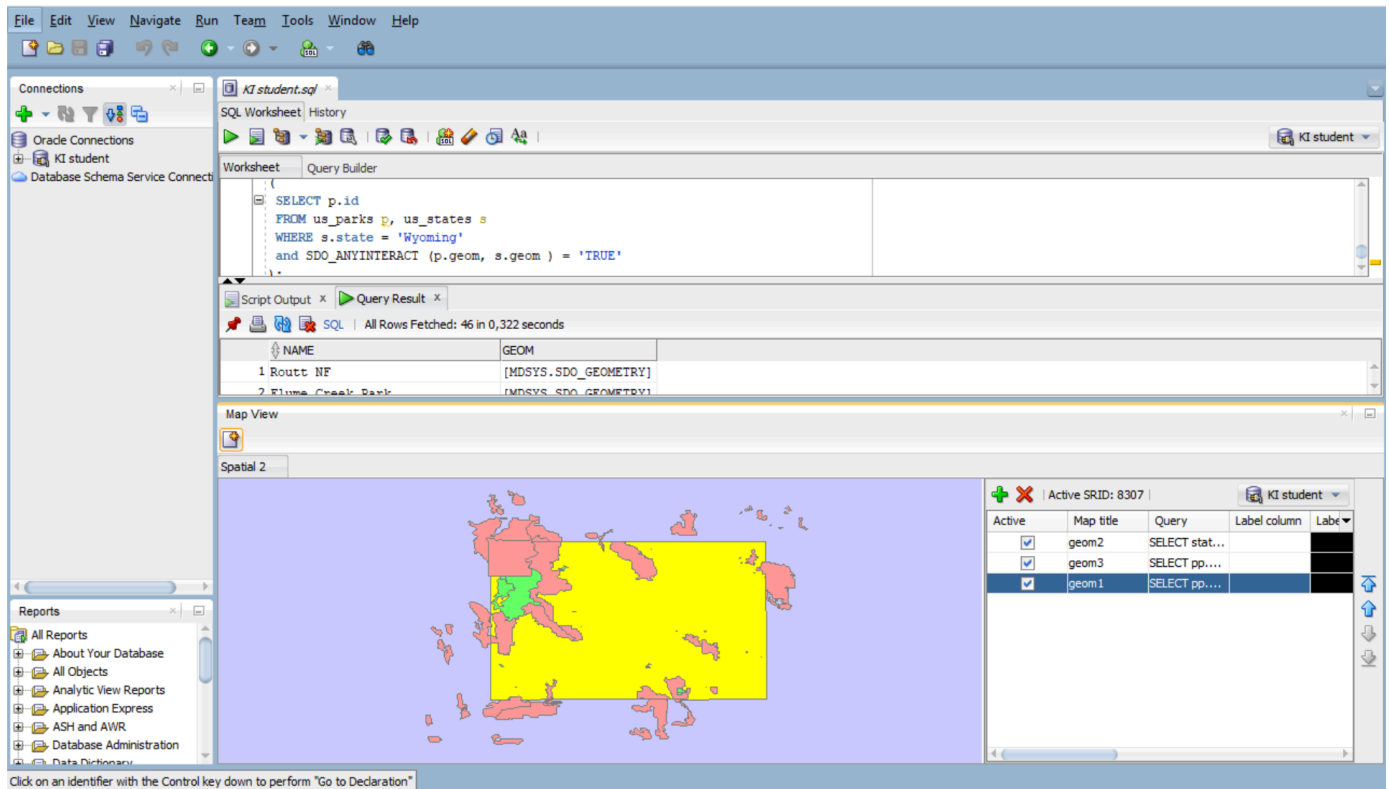


Porównaj wynik z:

```
SELECT p.name, p.geom
FROM us_parks p, us_states s
WHERE s.state = 'Wyoming'
AND SDO_ANYINTERACT (p.geom, s.geom ) = 'TRUE'
```

W celu wizualizacji użyj podzapytania

```
SELECT pp.name, pp.geom FROM us_parks pp
WHERE id IN
(
  SELECT p.id
  FROM us_parks p, us_states s
  WHERE s.state = 'Wyoming'
  and SDO_ANYINTERACT (p.geom, s.geom ) = 'TRUE'
)
```



SDO_ANYINTERACT zwraca szerszy zbiór wyników, również te parki, które przecinają granicę ze stanem Wyoming, podczas gdy SDO_INSIDE ogranicza wynik tylko do obiektów wewnętrznych.

Zadanie 4

Znajdź wszystkie jednostki administracyjne (us_counties) wewnątrz stanu New Hampshire

```
SELECT c.county, c.state_abrv, c.geom
FROM us_counties c, us_states s
WHERE s.state = 'New Hampshire'
AND SDO_RELATE ( c.geom,s.geom, 'mask=INSIDE+COVEREDBY') = 'TRUE';

SELECT c.county, c.state_abrv, c.geom
FROM us_counties c, us_states s
WHERE s.state = 'New Hampshire'
AND SDO_RELATE ( c.geom,s.geom, 'mask=INSIDE') = 'TRUE';

SELECT c.county, c.state_abrv, c.geom
FROM us_counties c, us_states s
WHERE s.state = 'New Hampshire'
AND SDO_RELATE ( c.geom,s.geom, 'mask=COVEREDBY') = 'TRUE';
```

W przypadku wykorzystywania narzędzia SQL Developer, w celu wizualizacji danych na mapie należy użyć podzapytania (podobnie jak w poprzednim zadaniu)

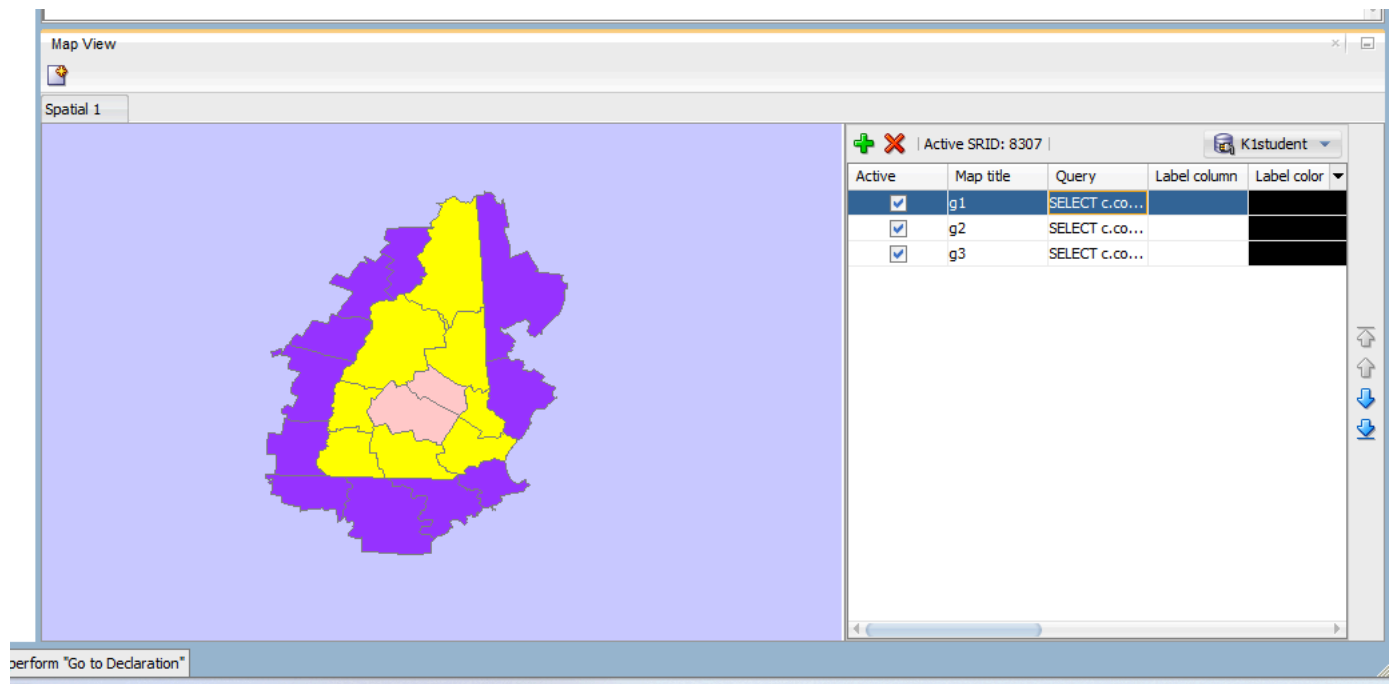
Wyniki, zrzut ekranu, komentarz

```
--- map g1
SELECT c.county,
       c.state_abrv,
       c.geom
FROM us_counties c
WHERE c.id IN
(
  SELECT c2.id
```

```
FROM us_counties c2,
     us_states s
WHERE s.state = 'New Hampshire'
     AND SDO_RELATE(
         c2.geom,
         s.geom,
         'mask=INSIDE+COVEREDBY'
     ) = 'TRUE'
);

--- map g2
SELECT c.county,
       c.state_abrv,
       c.geom
FROM us_counties c
WHERE c.id IN
(
    SELECT c2.id
    FROM us_counties c2,
         us_states s
    WHERE s.state = 'New Hampshire'
         AND SDO_RELATE(
             c2.geom,
             s.geom,
             'mask=INSIDE'
         ) = 'TRUE'
);

--- map g3
SELECT c.county,
       c.state_abrv,
       c.geom
FROM us_counties c
WHERE c.id IN
(
    SELECT c2.id
    FROM us_counties c2,
         us_states s
    WHERE s.state = 'New Hampshire'
         AND SDO_RELATE(
             c2.geom,
             s.geom,
             'mask=TOUCH'
         ) = 'TRUE'
);
```

Zapytanie g1 zwraca wszystkie hrabstwa znajdujące się w New Hampshire - całkowicie wewnętrzne (INSIDE) oraz te, które mogą leżeć na granicy stanu (COVEREDBY).

Zapytanie g2 zawiera tylko te hrabstwa, które w pełni są zawarte wewnątrz stanu New Hampshire.

Zapytanie g3 zwraca hrabstwa, które mają kontakt graniczny ze stanem New Hampshire - stykają się z granicą stanu, ale nie mają wspólnego wnętrza.

Zadanie 5

Znajdź wszystkie miasta w odległości 50 mili od drogi (us_interstates) I4

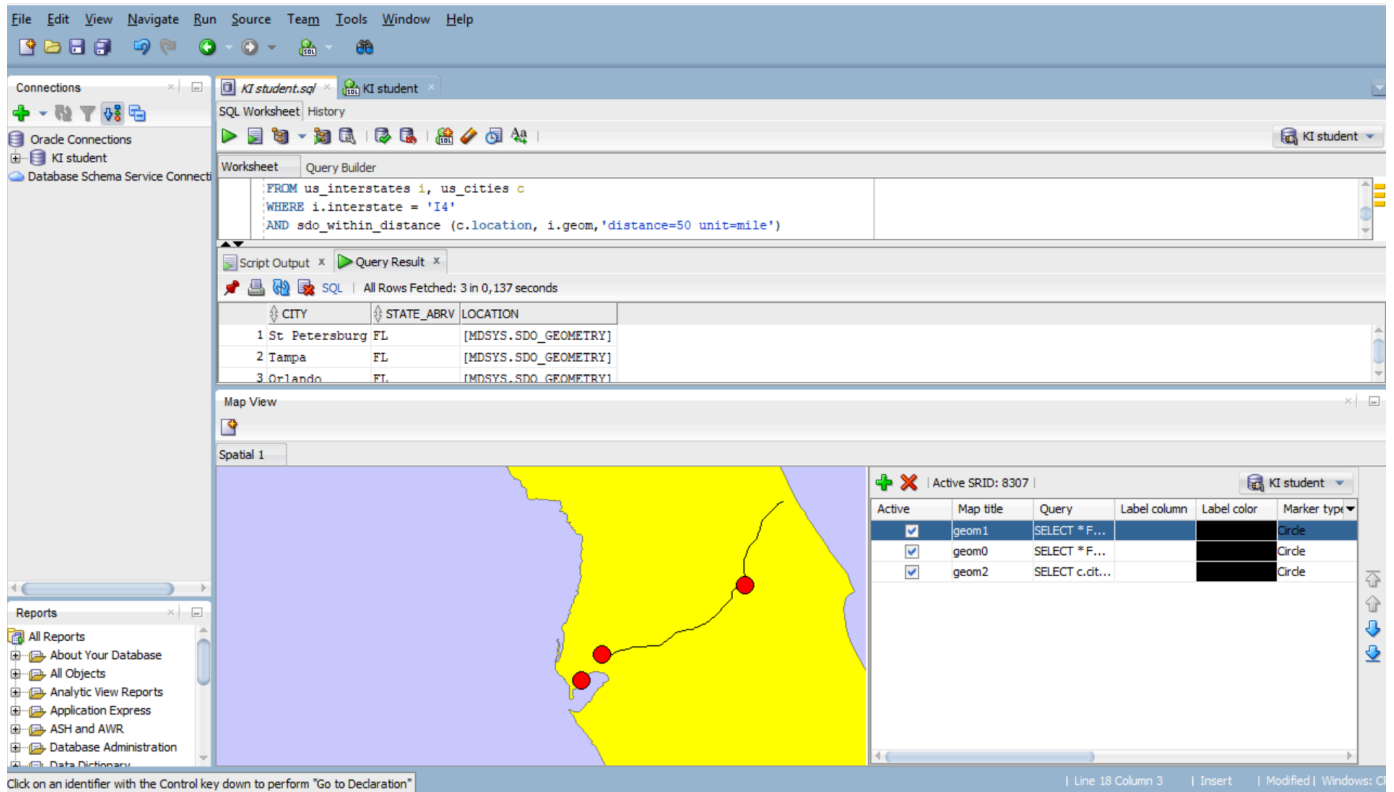
Pokaż wyniki na mapie

```
SELECT * FROM us_interstates
WHERE interstate = 'I4'

SELECT * FROM us_states
WHERE state_abrv = 'FL'

SELECT c.city, c.state_abrv, c.location
FROM us_cities c
WHERE ROWID IN
(
SELECT c.rowid
FROM us_interstates i, us_cities c
WHERE i.interstate = 'I4'
AND sdo_within_distance (c.location, i.geom, 'distance=50 unit=mile') = 'TRUE'
)
```

Wyniki, zrzut ekranu, komentarz



SDO_WITHIN_DISTANCE wyszukuje obiekty znajdujące się w zadanym promieniu od geometrii bazowej. Użycie podzapytania z ROWID pozwala na poprawne wyświetlenie warstwy w Map View.

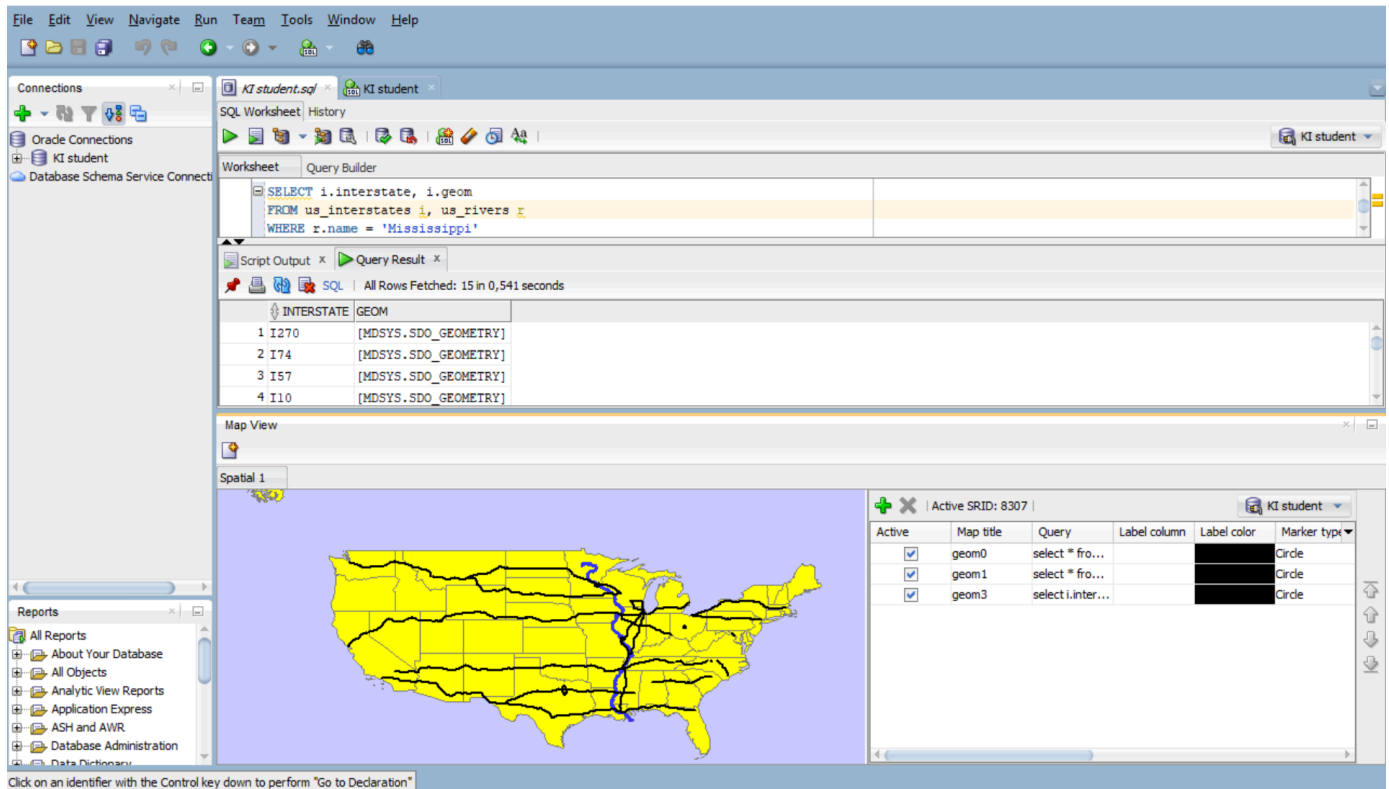
Dodatkowo:

a) Znajdź wszystkie drogi które przecinają rzekę

```
SELECT * FROM us_states

SELECT * FROM us_rivers
WHERE name = 'Mississippi'

SELECT i.interstate, i.geom
FROM us_interstates i
WHERE ROWID IN
(
  SELECT i.rowid
  FROM us_rivers r, us_interstates i
  WHERE r.name = 'Mississippi'
  AND SDO_ANYINTERACT (i.geom, r.geom) = 'TRUE'
)
```



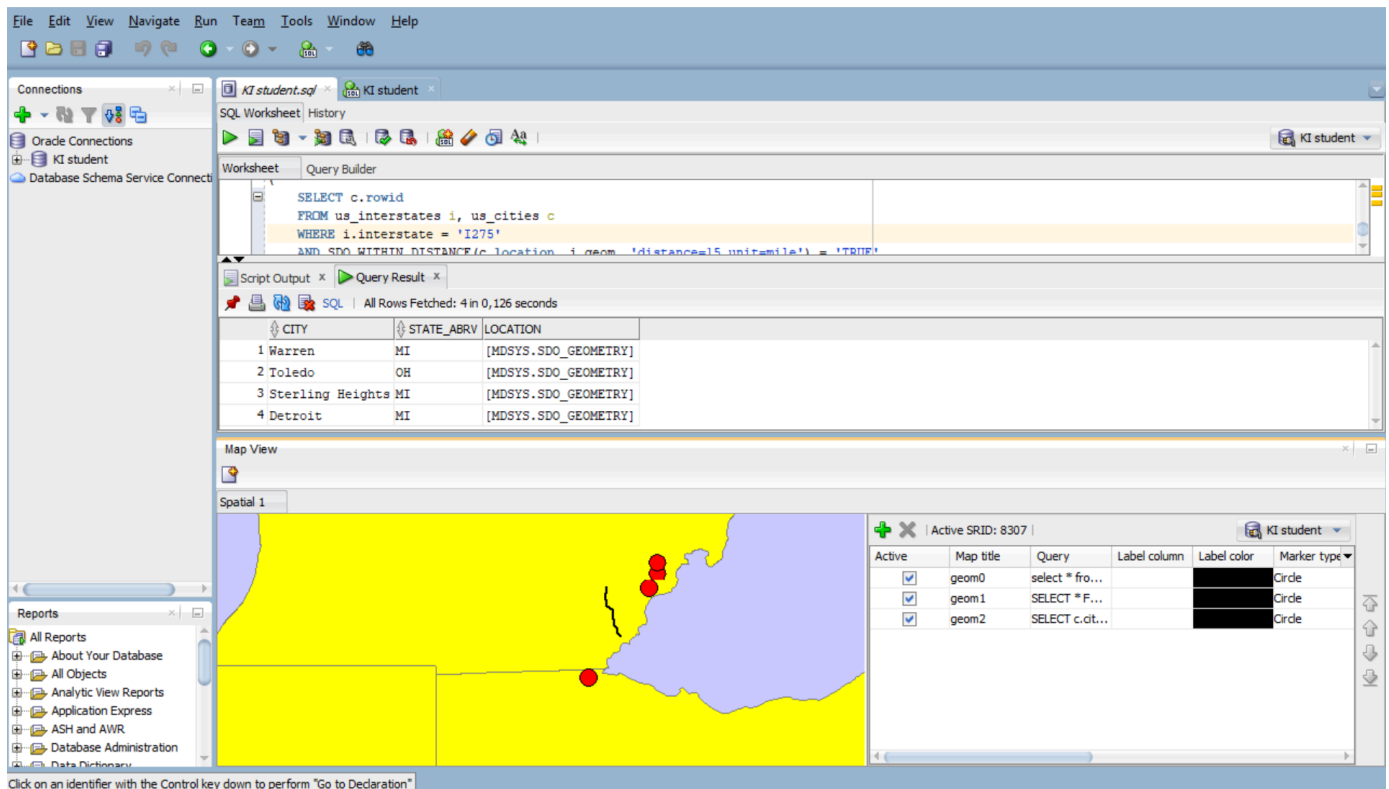
SDO_ANYINTERACT sprawdza czy obiekty wchodzą w jakąkolwiek relację przestrzenną, w tym przypadku szukamy dróg, które fizycznie przecinają linię rzeki.

b) Znajdź wszystkie miasta w odległości od 15 do 30 mil od drogi 'I275'

```
SELECT * FROM us_states

SELECT * FROM us_interstates
WHERE interstate = 'I275'

SELECT c.city, c.state_abrv, c.location
FROM us_cities c
WHERE ROWID IN
(
  SELECT c.rowid
  FROM us_interstates i, us_cities c
  WHERE i.interstate = 'I275'
  AND SDO_WITHIN_DISTANCE(c.location, i.geom, 'distance=30 unit=mile') = 'TRUE'
)
AND ROWID NOT IN
(
  SELECT c.rowid
  FROM us_interstates i, us_cities c
  WHERE i.interstate = 'I275'
  AND SDO_WITHIN_DISTANCE(c.location, i.geom, 'distance=15 unit=mile') = 'TRUE'
)
```



Połączenie dwóch warunków SDO_WITHIN_DISTANCE przy pomocy IN oraz NOT IN pozwala wyznaczyć obszar, w którym wykluczamy miasta leżące bliżej niż 15 mil od autostrady.

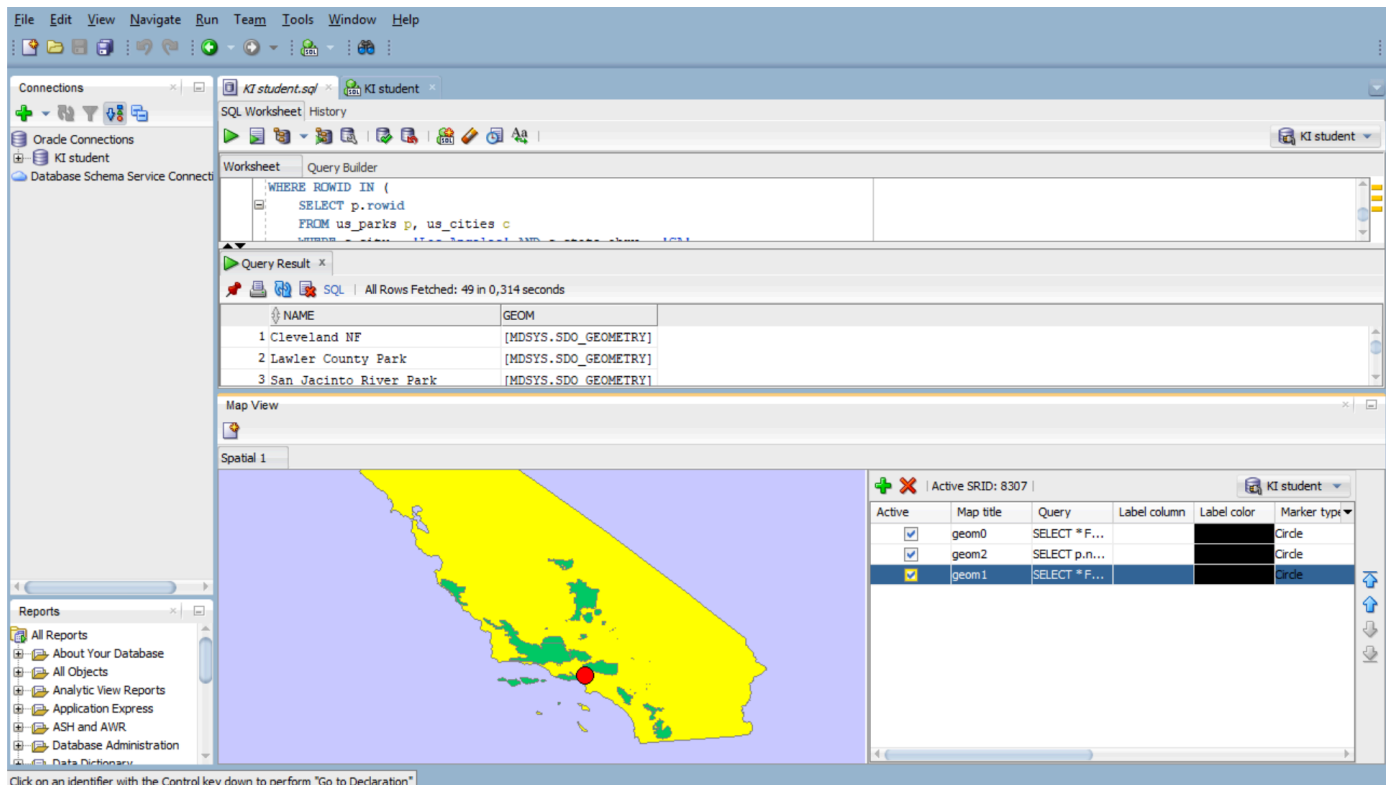
c) ltp. (własne przykłady)

Znajdziemy wszystkie parki znajdujące się w promieniu 100 mil od Los Angeles.

```
SELECT * FROM us_states
WHERE state_abrv = 'CA'

SELECT * FROM us_cities
WHERE city = 'Los Angeles'
AND state_abrv = 'CA'

SELECT p.name, p.geom
FROM us_parks p
WHERE ROWID IN (
  SELECT p.rowid
  FROM us_parks p, us_cities c
  WHERE c.city = 'Los Angeles' AND c.state_abrv = 'CA'
  AND SDO_WITHIN_DISTANCE(p.geom, c.location, 'distance=100 unit=mile') = 'TRUE'
)
```



SDO_WITHIN_DISTANCE wyznacza odległość między geometrią punktową (miasto) a poligonową (park).

Zadanie 6

Znajdź 5 miast najbliższych drogi I4

```
SELECT c.city, c.state_abrv, c.location
FROM us_interstates i, us_cities c
WHERE i.interstate = 'I4'
AND sdo_nn(c.location, i.geom, 'sdo_num_res=5') = 'TRUE';
```

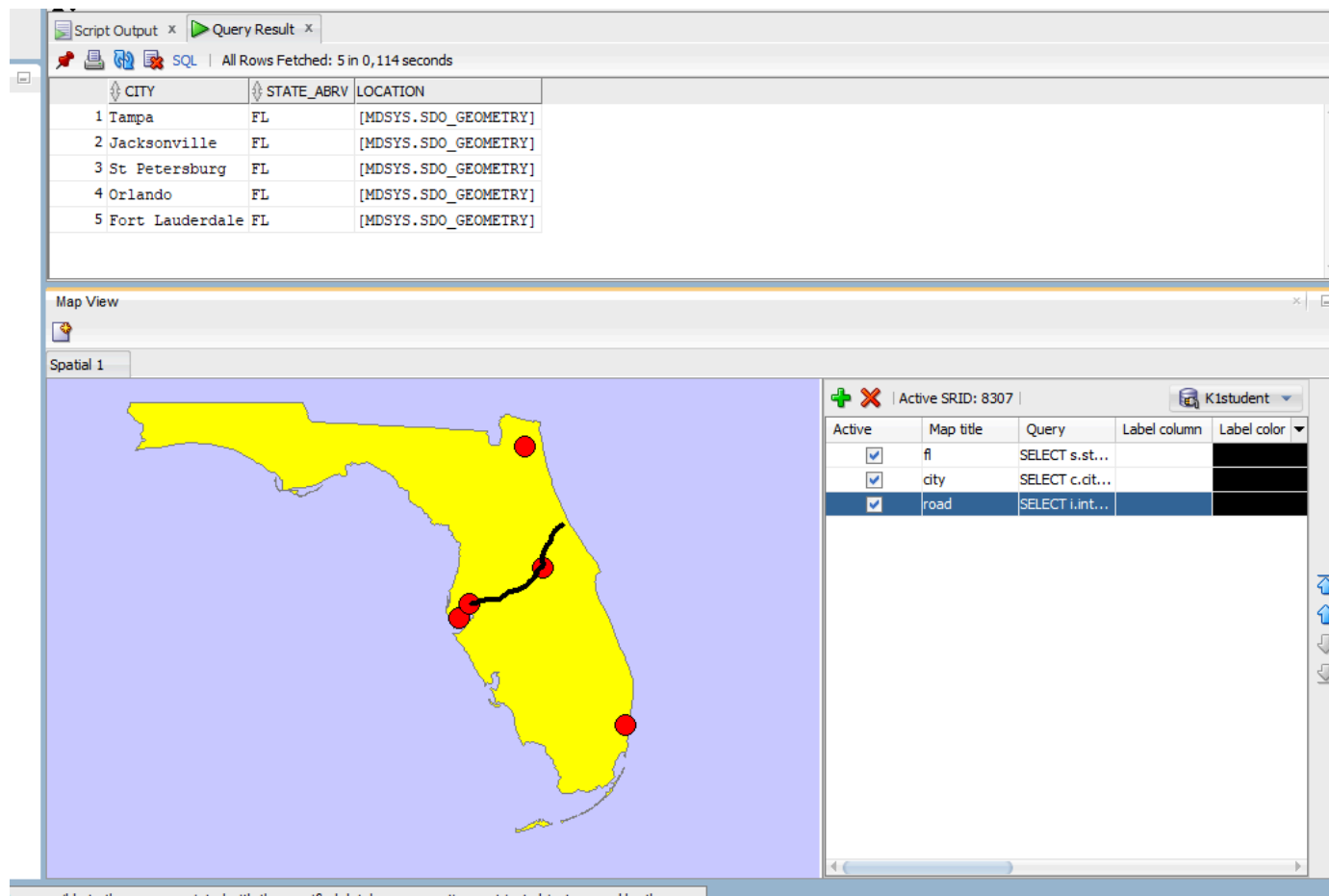
Wyniki, zrzut ekranu, komentarz

```
--- map fl
SELECT s.state,
       s.geom
FROM us_states s
WHERE s.state = 'Florida';

--- map city
SELECT c.city,
       c.state_abrv,
       c.location
FROM us_cities c
WHERE c.id IN
(
  SELECT c2.id
  FROM us_cities c2,
       us_interstates i
  WHERE i.interstate = 'I4'
        AND SDO_NN(c2.location, i.geom, 'sdo_num_res=5') = 'TRUE'
);

--- road
SELECT i.interstate,
       i.geom
```

```
FROM us_interstates i
WHERE i.interstate = 'I4';
```



SDO_NN - znajdź najbliższe obiekty geograficzne

Dodatkowo:

a) Podaj 3 parki narodowe do których jest najbliżej z Nowego Jorku, oblicz odległości do tych parków

```
SELECT p.name,
       p.geom,
       SDO_NN_DISTANCE(1) AS distance_km
FROM us_parks p,
     us_cities c
WHERE c.city = 'New York'
      AND c.state_abrv = 'NY'
      AND SDO_NN(
        p.geom,
        c.location,
        'sdo_num_res=3 unit=km',
        1
      ) = 'TRUE'
ORDER BY distance_km;
```

	NAME	GEOM	DISTANCE_KM
1	Institute Park	[MDSYS.SDO_GEOMETRY]	1,53989392335604
2	Prospect Park	[MDSYS.SDO_GEOMETRY]	1,71806926034585
3	Thompkins Park	[MDSYS.SDO_GEOMETRY]	2,13555672310317

Map View

Spatial 1

b) Znajdz 5 najbliższych dużych miast (o populacji powyżej 300 tys) od drogi 'I170'

```
SELECT *
FROM (
  SELECT c.city,
         c.state_abrv,
         c.pop90,
         c.location
  FROM us_cities c,
       us_interstates i
  WHERE i.interstate = 'I170'
        AND c.pop90 > 300000
        AND SDO_NN(c.location, i.geom) = 'TRUE'
)
WHERE ROWNUM <= 5;
```

	CITY	STATE_ABRV	POP90	LOCATION
1	St Louis	MO	396685	[MDSYS.SDO_GEOMETRY]
2	Kansas City	MO	435146	[MDSYS.SDO_GEOMETRY]
3	Indianapolis	IN	741952	[MDSYS.SDO_GEOMETRY]
4	Memphis	TN	610337	[MDSYS.SDO_GEOMETRY]
5	Chicago	IL	2783726	[MDSYS.SDO_GEOMETRY]

Map View

WHERE ROWNUM <= 5 jako znane LIMIT 5, żeby ograniczyć liczbę wyświetlanych wyników. Jeżeli damy ograniczenie do środka SDO_NN to potem sprawdzany jest warunek populacji przez co zmniejsza się liczba wierszy w wyniku - nie otrzymujemy 5.

c) ltp. (własne przykłady).

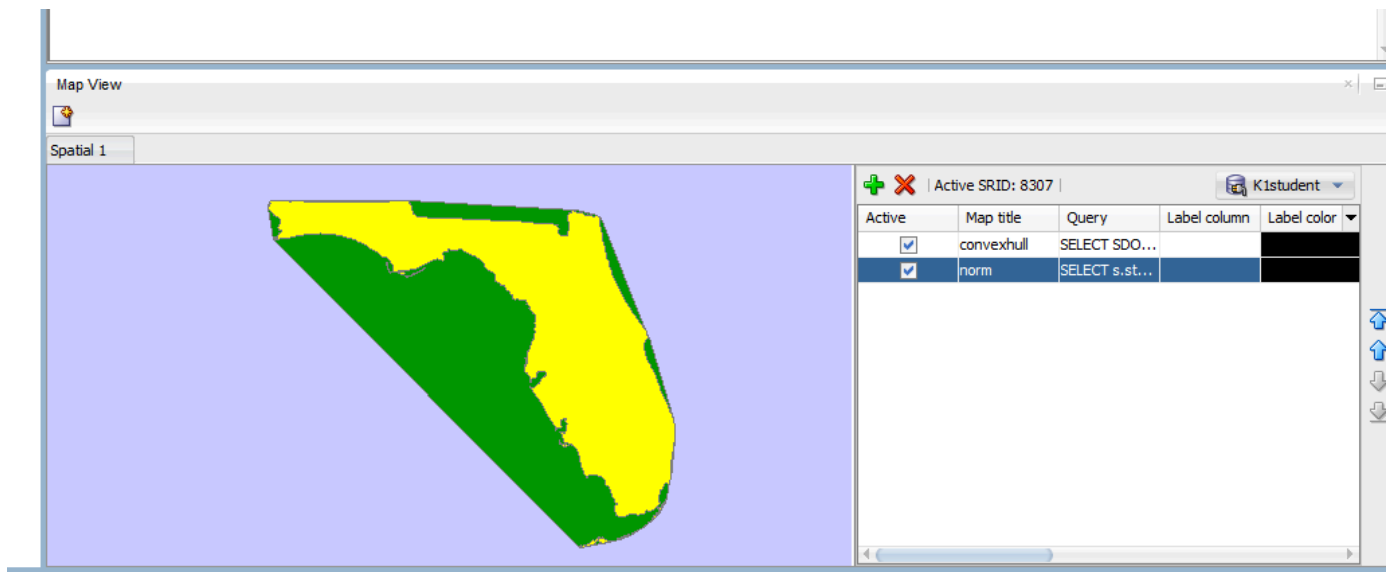
- np. przetestuj działanie funkcji
 - sdo_intersection, sdo_union, sdo_difference
 - sdo_buffer
 - sdo_centroid, sdo_mbr, sdo_convexhull, sdo_simplify

Wyniki, zrzut ekranu, komentarz (dla każdego z podpunktów)

Wyniki nowych funkcji porównane z oryginalnymi granicami Florydy.

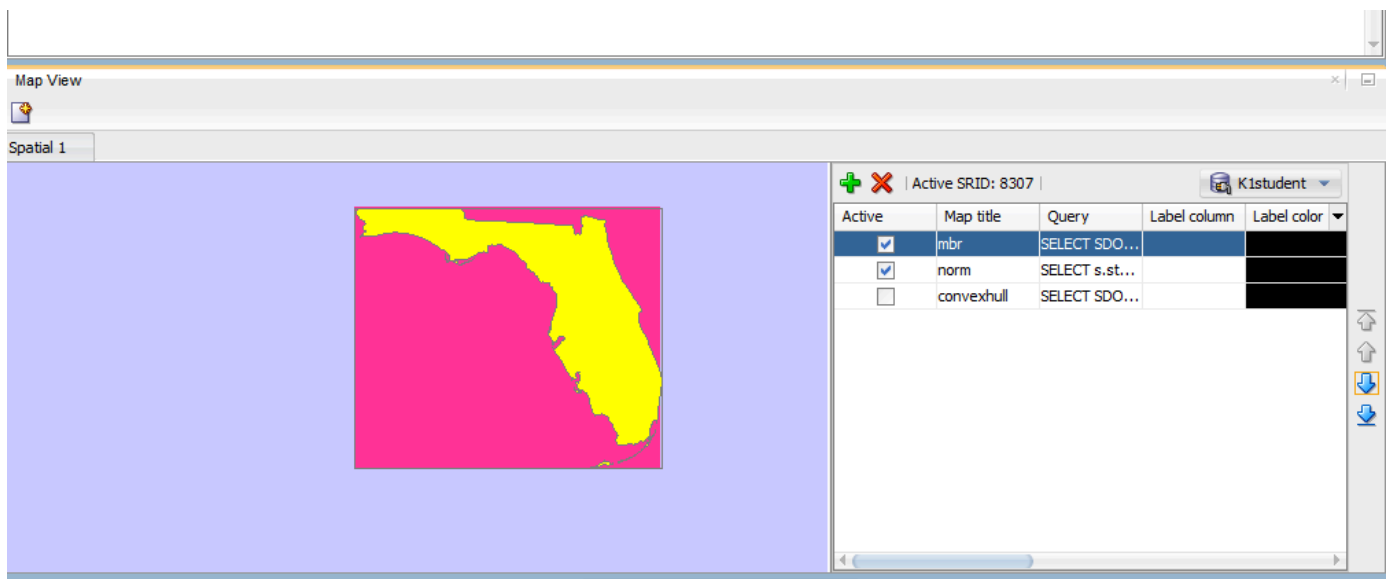
```
SELECT SDO_GEOM.SDO_CONVEXHULL(geom, 0.005) AS geom
FROM us_states
```

```
WHERE state = 'Florida';
```



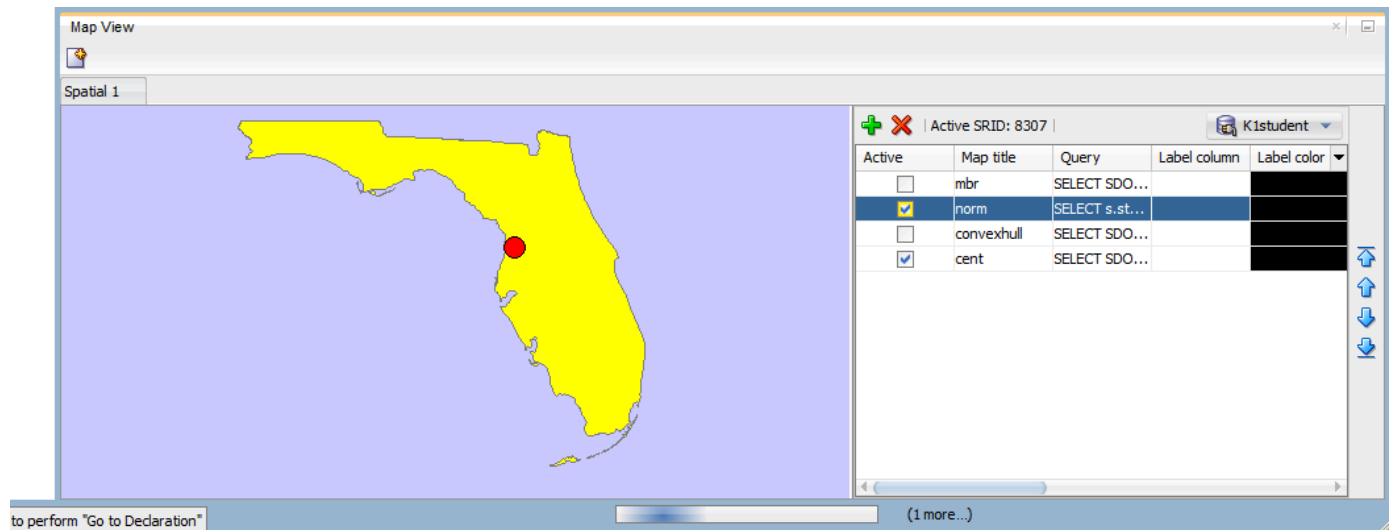
SDO_CONVEXHULL tworzy wypukłą otoczkę Florydy, czyli najmniejszy wypukły wielokąt zawierający cały stan.

```
SELECT SDO_GEOM.SDO_MBR(geom) AS geom
FROM us_states
WHERE state = 'Florida';
```



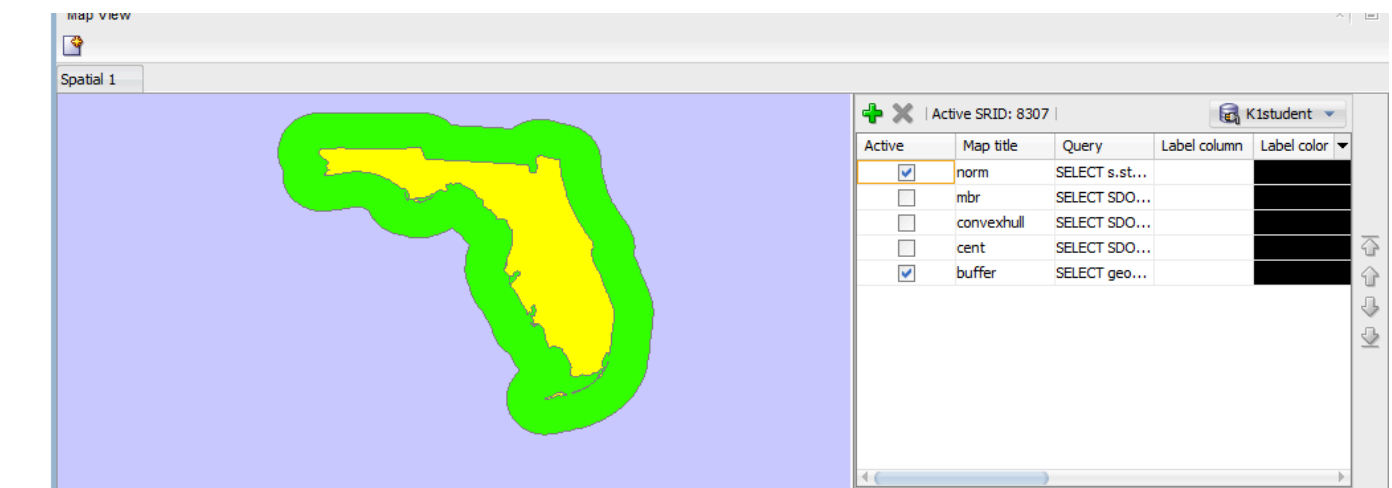
SDO_MBR zwraca minimalny prostokąt obejmujący całą Florydę.

```
SELECT SDO_GEOM.SDO_CENTROID(s.geom, 0.005) AS geom
FROM us_states s
WHERE state = 'Florida';
```

SDO_CENTROID wyznacza geometryczny środek Florydy.

```
SELECT geom
FROM (
  SELECT SDO_GEOM.SDO_BUFFER(s.geom, 100000, 0.005) AS geom
  FROM us_states s
  WHERE s.state = 'Florida'
)
```



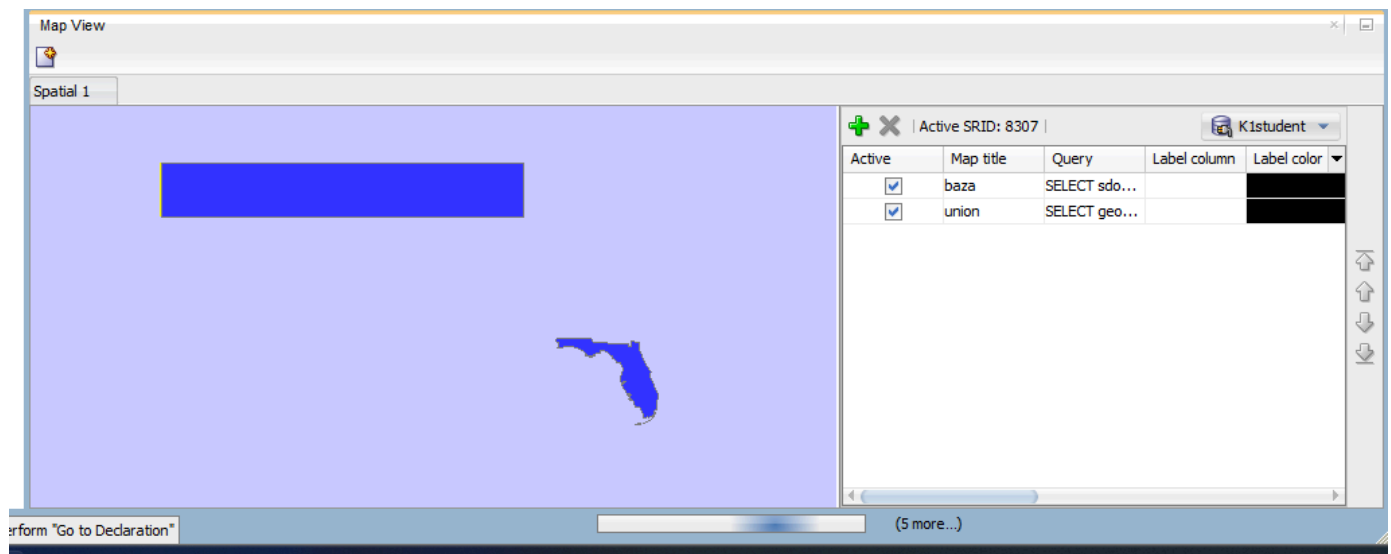
SDO_BUFFER tworzy strefę buforową wokół Florydy w zadanej odległości - w przykładzie wybrana duża wartość, żeby była dobrze widoczna.

Teraz będziemy porównywać wyniki zapytań między zadaną geometrią a wybranym stanem.

```

SELECT geom
FROM (
  SELECT SDO_GEOM.SDO_UNION(
    s.geom,
    sdo_geometry(
      2003, 8307, NULL,
      sdo_elem_info_array(1,1003,3),
      sdo_ordinate_array(-117.0, 40.0, -90.0, 44.0)
    ),
    0.005
  ) AS geom
FROM us_states s
WHERE s.state = 'Florida'
)

```

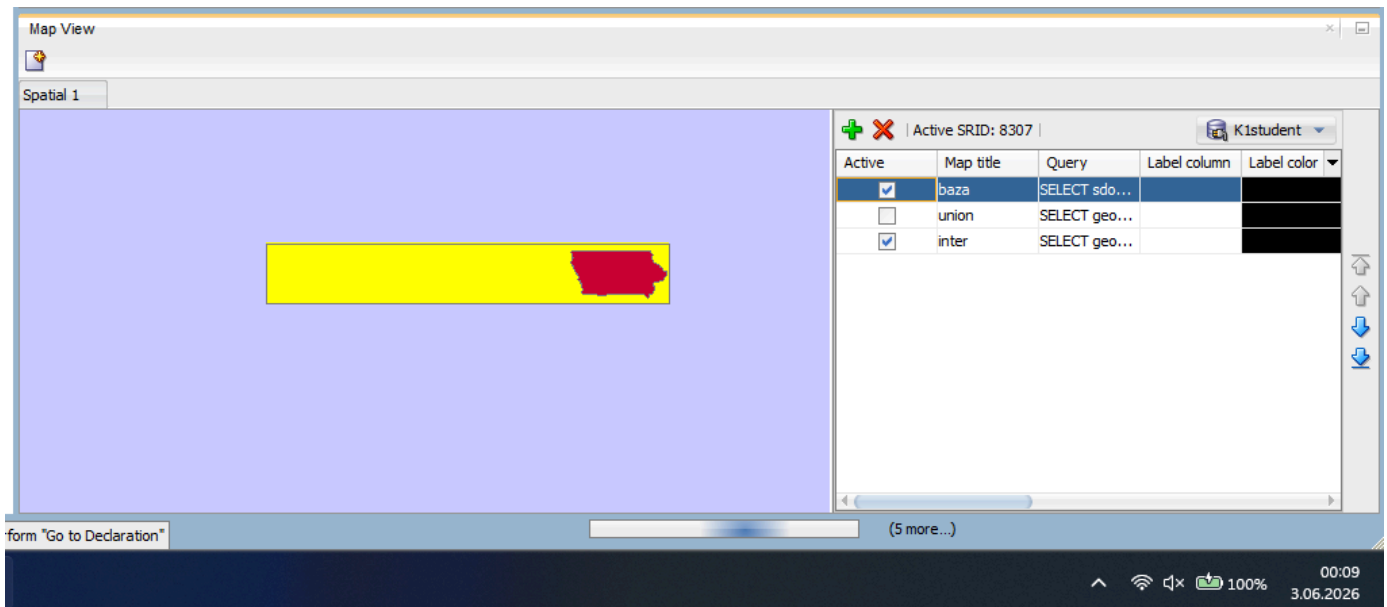


SDO_UNION łączy geometrię stanu Florida z zadany prostokątem. Wynikiem jest jeden obiekt, który obejmuje oba obszary.

```

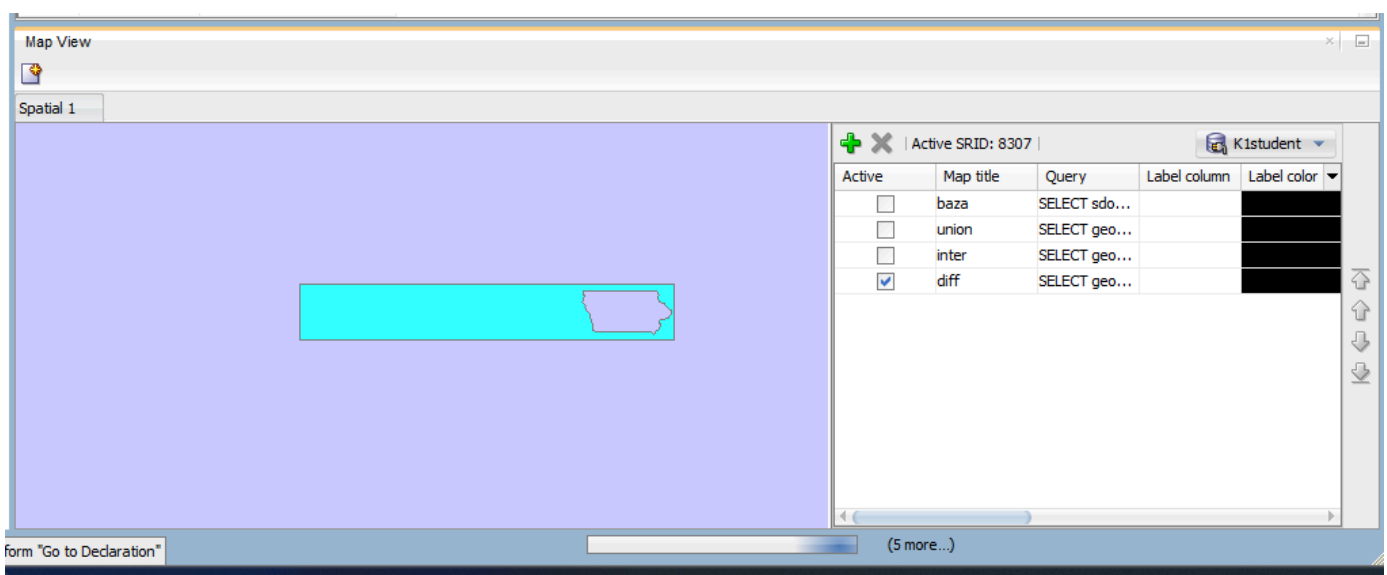
SELECT geom
FROM (
  SELECT SDO_GEOM.SDO_INTERSECTION(
    s.geom,
    sdo_geometry(
      2003, 8307, NULL,
      sdo_elem_info_array(1,1003,3),
      sdo_ordinate_array(-117.0, 40.0, -90.0, 44.0)
    ),
    0.005
  ) AS geom
FROM us_states s
WHERE s.state = 'Iowa'
)

```



SDO_INTERSECTION zwraca wspólną część geometrii lowy oraz prostokąta - pokazuje tylko ten fragment Iowa, który znajduje się w zadanym obszarze. Żółty obszar pokazany dla referencji.

```
SELECT geom
FROM (
  SELECT SDO_GEOM.SDO_DIFFERENCE(
    sdo_geometry(
      2003, 8307, NULL,
      sdo_elem_info_array(1,1003,3),
      sdo_ordinate_array(-117.0, 40.0, -90.0, 44.0)
    ),
    s.geom,
    0.005
  ) AS geom
  FROM us_states s
  WHERE s.state = 'Iowa'
)
```



SDO_DIFFERENCE pokazuje obszar prostokąta minus lowy.

Zadanie 7

Wykonaj kilka własnych przykładów/analiz

Wyniki, zrzut ekranu, komentarz

Wizualizacje i analizy w Jupyter Notebook'u. Połączenie z bazą:

```
import oracledb
import folium
import geojson

cs = oracledb.makedsn("dbmanage.lab.ii.agh.edu.pl", 1521, sid="DBMANAGE")
c = oracledb.connect(user="student", password="stu638dent", dsn=cs)
cursor = c.cursor()
cursor.execute("ALTER SESSION SET CURRENT_SCHEMA = US_SPAT")
```

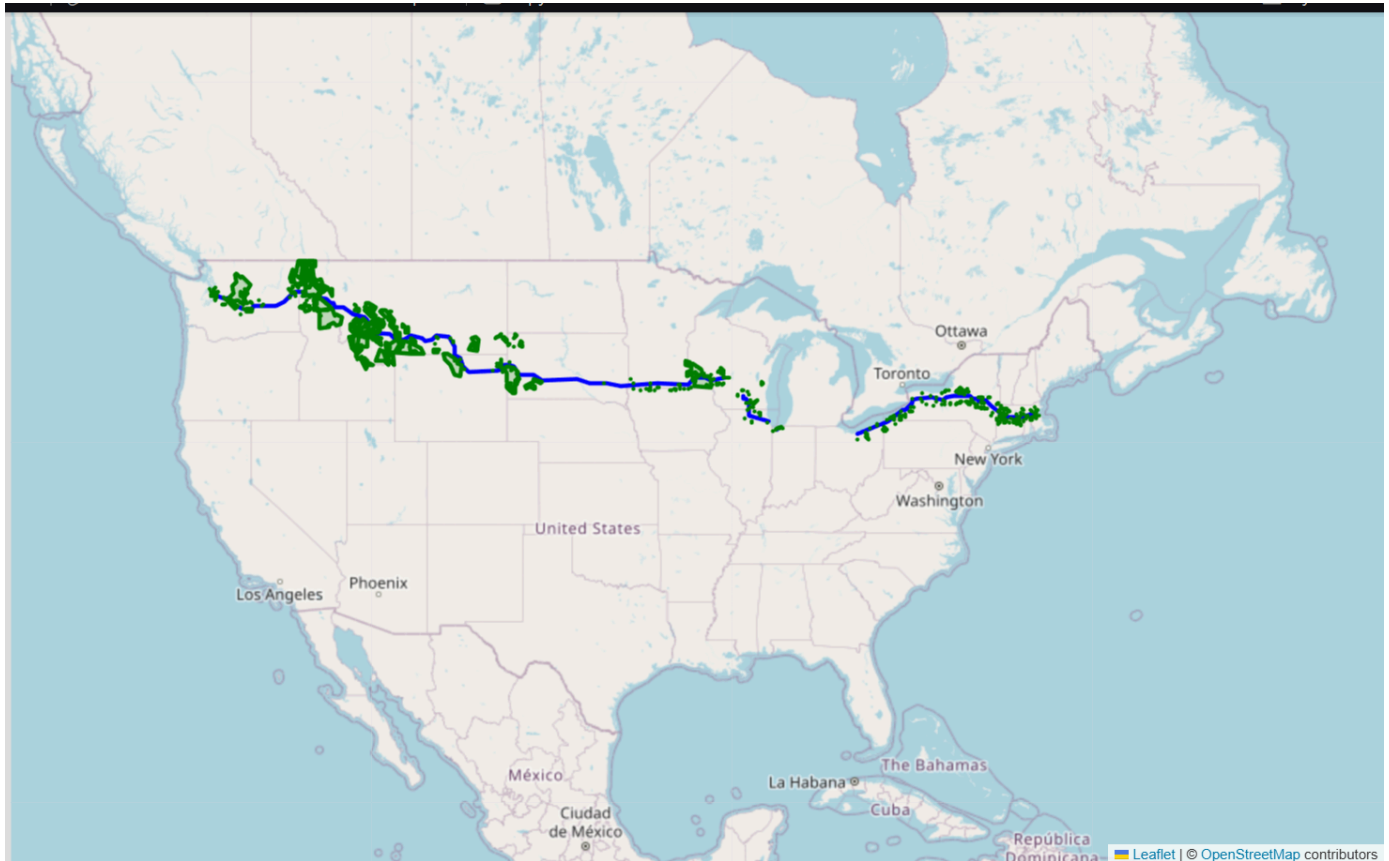
a) Parki znajdujące się do 20 mil od autostrady I90

```
m1 = folium.Map(location=[44, -100], zoom_start=4)

q_road = "SELECT sdo_util.to_geojson(geom) FROM us_interstates WHERE interstate = 'I90'"
r_road = cursor.execute(q_road).fetchall()
f_road = [geojson.Feature(geometry=geojson.loads(row[0].read())) for row in r_road]
folium.GeoJson(geojson.FeatureCollection(f_road), style_function=lambda x: {'color': 'blue'}).add_to(m1)

q_parks = """
SELECT sdo_util.to_geojson(p.geom)
FROM us_parks p, us_interstates i
WHERE i.interstate = 'I90'
AND SDO_WITHIN_DISTANCE(p.geom, i.geom, 'distance=20 unit=mile') = 'TRUE'
"""
r_parks = cursor.execute(q_parks).fetchall()
f_parks = [geojson.Feature(geometry=geojson.loads(row[0].read())) for row in r_parks]
folium.GeoJson(geojson.FeatureCollection(f_parks), style_function=lambda x: {'color':
'green'}).add_to(m1)

m1
```



SDO_WITHIN_DISTANCE wyszukuje parki w promieniu 20 mil od autostrady. Funkcja sdo_util.to_geojson konwertuje geometrię prosto z bazy na format czytelny dla biblioteki folium.

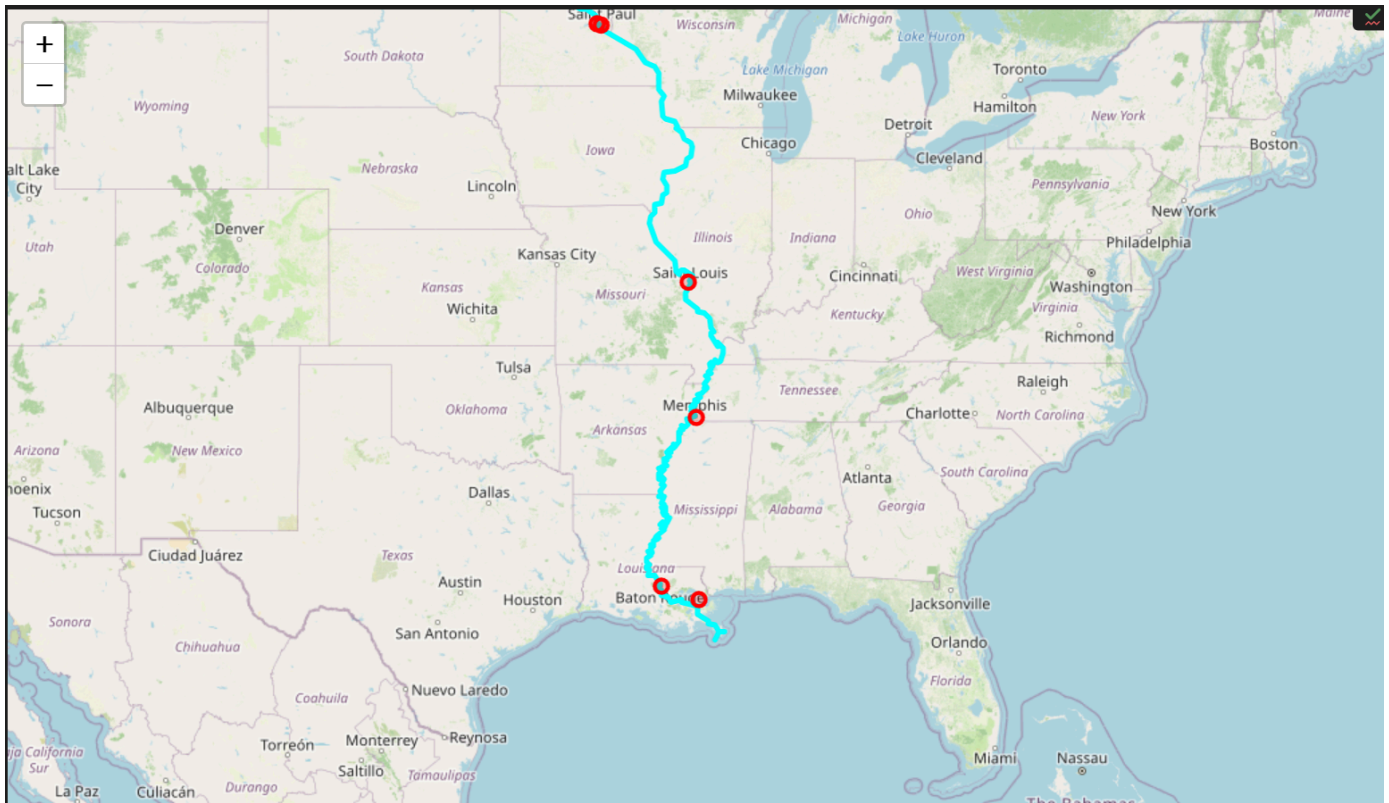
b) Miasta powyżej 100 tys. mieszkańców leżące w strefie do 10 mil od rzeki Mississippi

```
m2 = folium.Map(location=[35, -90], zoom_start=5)

q_river = "SELECT sdo_util.to_geojson(geom) FROM us_rivers WHERE name = 'Mississippi'"
r_river = cursor.execute(q_river).fetchall()
f_river = [geojson.Feature(geometry=geojson.loads(row[0].read())) for row in r_river]
folium.GeoJson(geojson.FeatureCollection(f_river), style_function=lambda x: {'color': 'cyan', 'weight': 4}).add_to(m2)

q_cities = """
SELECT sdo_util.to_geojson(c.location)
FROM us_cities c, us_rivers r
WHERE r.name = 'Mississippi' AND c.pop90 > 100000
AND SDO_WITHIN_DISTANCE(c.location, r.geom, 'distance=10 unit=mile') = 'TRUE'
"""
r_cities = cursor.execute(q_cities).fetchall()
f_cities = [geojson.Feature(geometry=geojson.loads(row[0].read())) for row in r_cities]
folium.GeoJson(geojson.FeatureCollection(f_cities), marker=folium.CircleMarker(radius=5, color='red', fill=True)).add_to(m2)
```

m2



Połączenie warunku na populację z operatorem SDO_WITHIN_DISTANCE. Miasta spełniające oba warunki zostały wyrenderowane punktowo przy użyciu znacznika CircleMarker na czerwono.

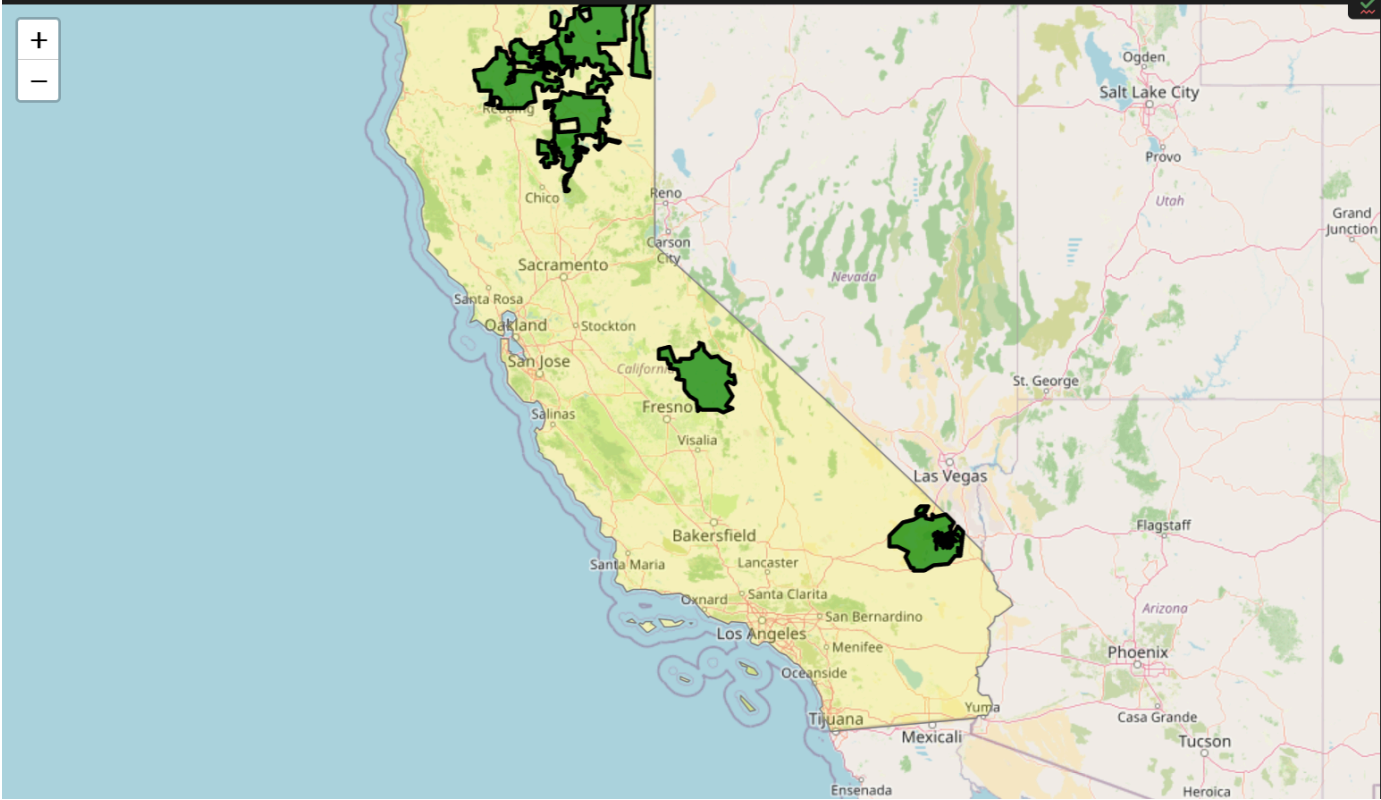
c) 5 największych parków wewnątrz stanu Kalifornia

```
m3 = folium.Map(location=[36, -119], zoom_start=6)

q_state = "SELECT sdo_util.to_geojson(geom) FROM us_states WHERE state_abrv = 'CA'"
f_state = [geojson.Feature(geometry=geojson.loads(row[0].read())) for row in
cursor.execute(q_state).fetchall()]
folium.GeoJson(geojson.FeatureCollection(f_state), style_function=lambda x: {'fillColor': 'yellow',
'color': 'gray', 'weight': 1}).add_to(m3)

q_top_parks = """
SELECT sdo_util.to_geojson(p.geom)
FROM us_parks p, us_states s
WHERE s.state_abrv = 'CA' AND SDO_INSIDE(p.geom, s.geom) = 'TRUE'
ORDER BY SDO_GEOM.SDO_AREA(p.geom, 0.005, 'unit=SQ_KM') DESC
FETCH FIRST 5 ROWS ONLY
"""
r_top = cursor.execute(q_top_parks).fetchall()
f_top = [geojson.Feature(geometry=geojson.loads(row[0].read())) for row in r_top]
folium.GeoJson(geojson.FeatureCollection(f_top), style_function=lambda x: {'fillColor': 'green', 'color':
'black', 'fillOpacity': 0.7}).add_to(m3)

m3
```



Wykorzystałmy SDO_GEOM.SDO_AREA do wyliczenia pola powierzchni w km². Pozwoliło to na przesortowanie wyników operatorem ORDER BY i zwrócenie 5 największych parków.

Punktacja

zad	pkt
1	0,5
2	0,5
3	0,5
4	0,5
5	1
6	2
7	2
razem	7